

SOP Technical API Test

API to check: <https://reqres.in/>

Important: Header **x-api-key** is needed for requests to work, value must be: **reqres-free-v1**

1. **Scenario:** The **/api/login** endpoint is used for user authentication and requires both email and password for successful login.

Objective: Test both successful and failed login attempts and ensure the API handles missing fields appropriately.

Task:

- Write a test that:
 1. Send a POST request with valid credentials
 2. Send a POST request with invalid credentials
 3. Do any other validations that you see fit related to responses and possible status codes.

API: [api/login](#)

Request example:

```
{
  "email": "eve.holt@reqres.in",
  "password": "cityslicka"
}
```

2. **Scenario:** The **/api/users/2** endpoint returns details of a specific user.

Objective: Validate the data structure and specific content of the response.

Task:

- Write a test that:
 1. Send a **GET** request to **/api/users/2** and verifies the response.
 2. Do any other validations that you may think of.

API: [api/users/2](#)

3. **Scenario:** The **/api/users** endpoint allows creating a user by sending a **POST** request with name and job fields.

Objective: Test the API's response to various invalid inputs when creating a user.

Task:

- Write a test that:
 1. Send a POST request with valid name and job fields (e.g., name: "John", job: "developer") and validates the response.
 2. Do any other validations that you may see fit.

API: [api/users](#)

Request example:

```
{
  "name": "John",
  "job": "developer"
}
```

4. Scenario: Validate pagination handling and the integrity of data across pages.

Objective: The `/api/users` endpoint returns paginated data, allowing clients to request different pages of users.

Task:

- Write a test that:
 1. Sends a GET request to `/api/users?page=2` and verifies the correct number of users is returned.
 2. Validates that the page, `per_page`, total, and `total_pages` fields in the response contain accurate values.
 3. Fetches page 1 and page 2 and ensures that the users from the two pages are unique (i.e., no user appears on both pages).

API: `api/users`

5. Scenario: The following endpoint allows you to delay the response.

Objective: Do validation as you see fit

Task:

- Write a test that:
 1. Do validation as you see fit

API: `api/users?delay=<number>`

6. Bonus Question 1: Chained requests for user list and specific user

Scenario: You want to first fetch a list of users and then retrieve detailed information about one of those users

Objective: Interact with the list of users and retrieve specific user details based on the list.

- Tasks:
 1. Have in mind clean code and reusability
 2. Do any other validations that you may think.

5. Bonus Question 2: Theory

If the token returned was meant to be used for authentication, how would you structure your tests to:

- Store and reuse tokens securely
- Add auth headers automatically
- Handle token expiration or reuse

A HTML report must be generated and attached to the zip.

P.D: Please use some programming language or framework to do this and avoid using postman, jmeter or sending a txt file.